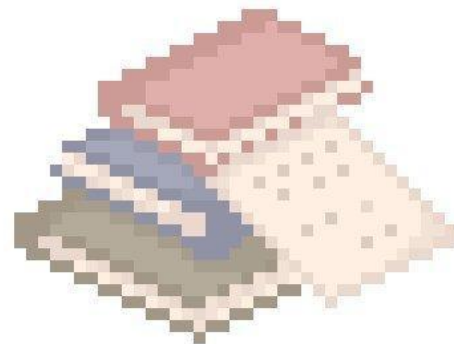


Numerai Example Scriptsを読み解く

Numerai Community Tokyo Meetup
2023/04/08

自己紹介

- Yuichiro Nishimoto
 - Twitter: [@nishimt_general](#)
 - Kaggle: [nishimoto](#)
 - Researchgate: [Yuichiro Nishimoto](#)
- 普段は企業でバイオ統計系の研究職
 - ソフトバンク 1年 → メタジェン 6年



目次

- はじめに
 - 前提知識
 - 今日の目的
- Numerai Example Scriptsについて
 - 概要
 - データ分割:Purged K-Fold
 - 学習:LightGBM
 - 学習:複数の目的変数の利用
 - 後処理:Feature Neutralization
 - 評価
- まとめ

はじめに – 前提知識

- 機械学習の基本的な知識

... Titanicは何も見ないで回せるくらい

- Numerai (Tournament) の基礎知識

... 1回はSubmitしている方が理解が進みやすいと思います

- [Ohoriさんが作ったNumeraiのgeneralな説明](#)
- [KatsuさんのV2データ用コード](#)
- [自分が作ったV4データ用のコード](#)

はじめに - 今日の目的

- 大目的: Numerai参加者を増やしたい!
 - Numeraiの情報はTCがある都合上、情報の共有がされづらい
 - NMRの価値が上がってほしい
- 目的: 「Tutorialコードを動かす」の次としてExample Scriptを広めたい
 - Numeraiの考えるNumerai tournamentの基礎が入っている

Numerai Example Scriptsとは？

- Numeraiが公式で公開しているExample scriptのこと

example-scripts/example_model_advanced.py at master · numerai/example-scripts

The official example scripts for the Numerai Data Science Tournament - example-scripts/example_model_advanced.py at master ·

https://github.com/numerai/example-scripts/blob/master/example_model_advanced.py

numerai/example-scripts

The official example scripts for the Numerai Data Science Tournament

41 Contributors 9 Issues 525 Stars 229 Forks

https://github.com/numerai/example-scripts/blob/master/example_model_advanced.py

- Diagnosticsにかけてみると、並大抵のモデルより成績がよい

Performance 📈		Risk 📉		Other 📊	
Sharpe Ratio	1.0564	Std. Dev.	0.0282	Autocorr	0.1286
CORR	0.0298	Feat. Exposure	0.1936	FNC	0.0267
FNCv3	0.0260	Max Drawdown	-0.1450	Ex. Preds Corr	0.6304

【Tips】

同リポジトリに置いてあるNotebookも有用な情報が色々載っています

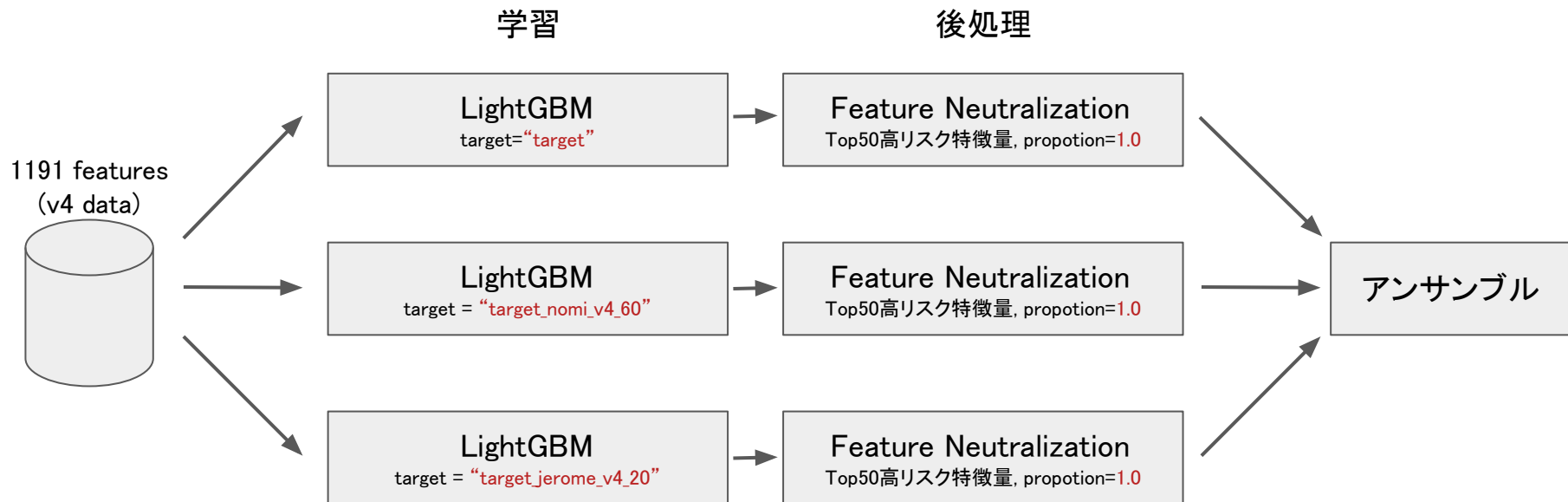
https://github.com/numerai/example-scripts/blob/master/analysis_and_tips.ipynb

目次

- はじめに
 - 前提知識
 - 今日の目的
- Numerai Example Scriptsについて
 - 概要
 - データ分割:Purged K-Fold
 - 学習:LightGBM
 - 学習:複数の目的変数の利用
 - 後処理:Feature Neutralization
 - 評価
- まとめ

Numerai Example Scripts – 概要

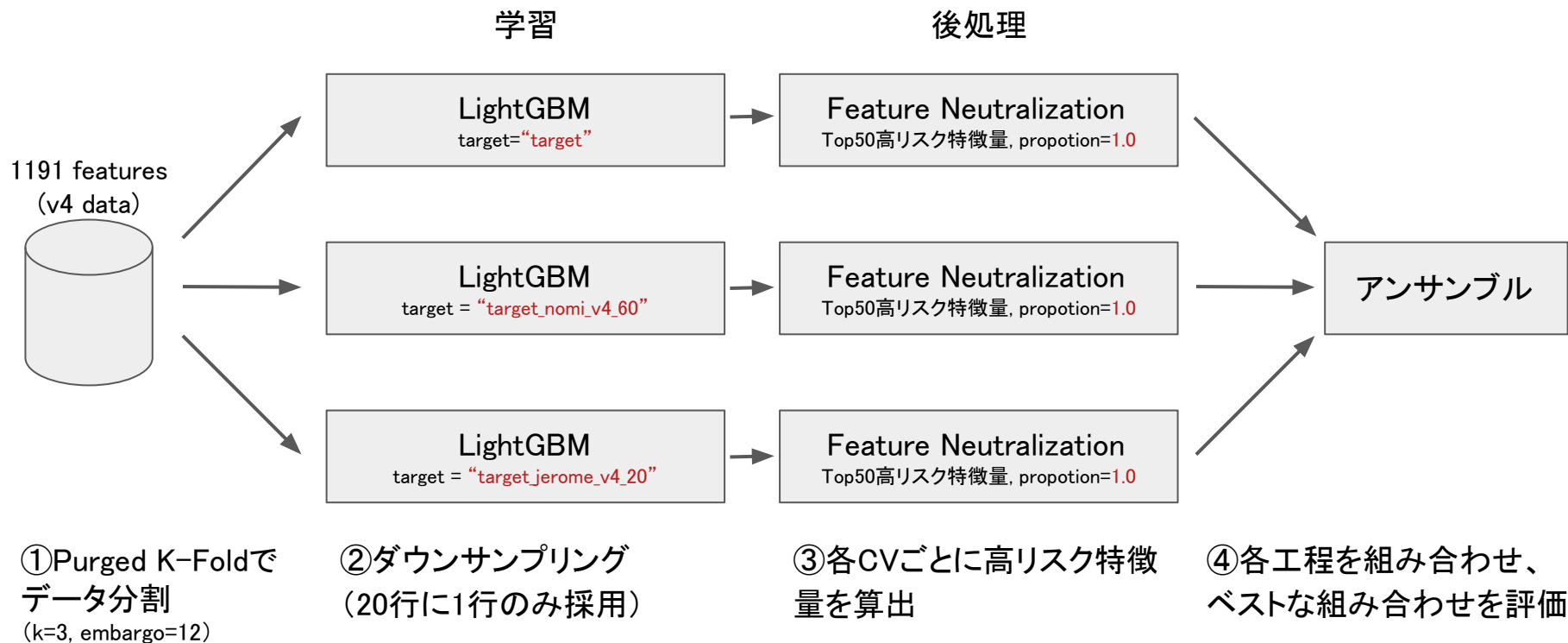
①モデル選択フェーズ、②実データ予測フェーズで細かい挙動が異なるものの大まかな流れは以下



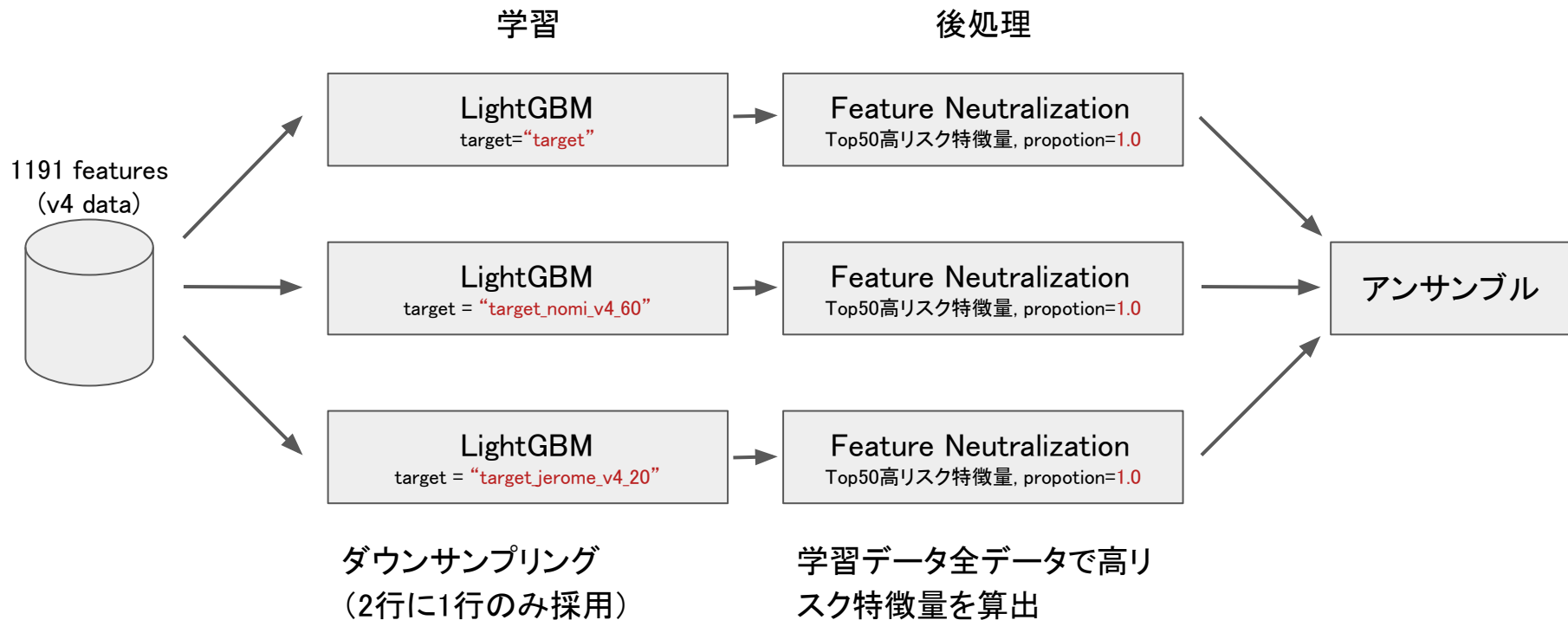
Kaggle上にて動かせるように置いてあります。

<https://www.kaggle.com/code/nishimoto/numerai-v4-data-example-script>

Numerai Example Scripts概要 - ①モデル選択



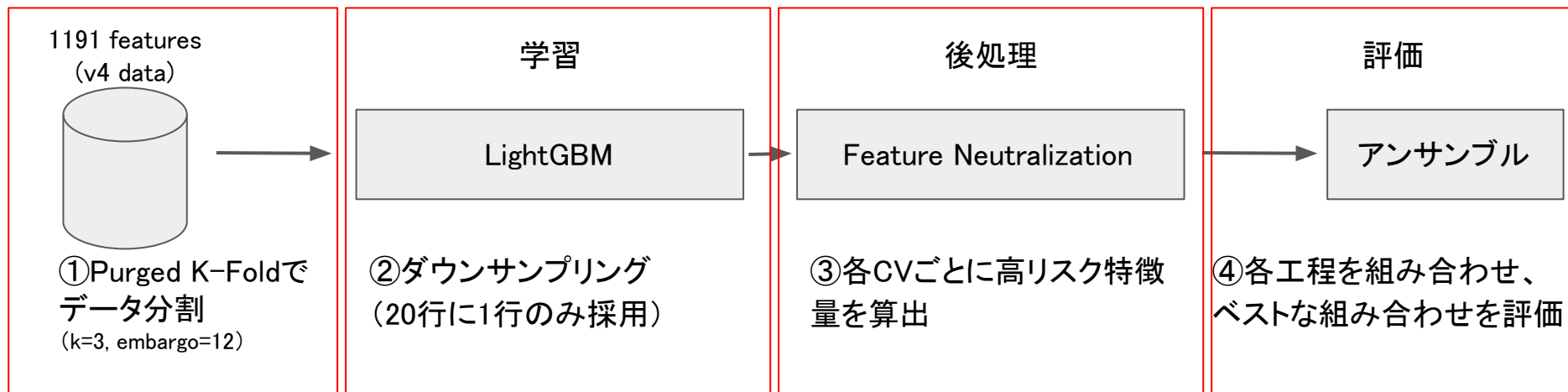
Numerai Example Scripts概要 - ②実データ予測



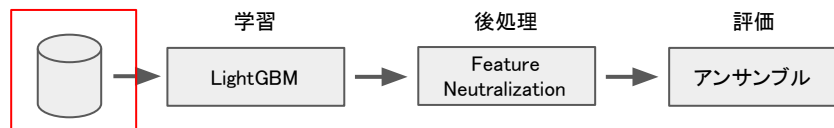
Numerai Example Scripts概要

11

以下の4工程について詳細を見ていく



Tips: メモリの節約

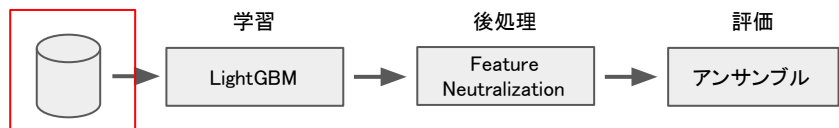


v3以降のデータは、デフォルトで省メモリデータ(int8データ)が提供されている。
何も損はないので使うべき。明示はされていないが、.parquetの前に _int8をつければOK。
ダウンロードするときは以下のコードをよく使用しています。

```
# redirect_stderr: tqdmを吐き出さないようにする
# from contextlib import redirect_stderr
# from numerapi import NumerAPI
```

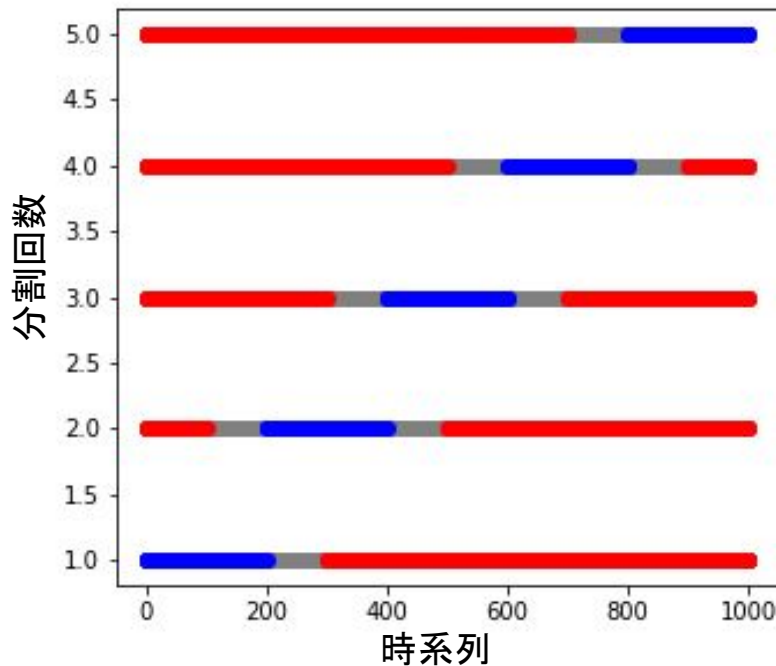
```
napi = NumerAPI()
with redirect_stderr(open(os.devnull, 'w')):
    napi.download_dataset("v4/train_int8.parquet", "train.parquet")
    napi.download_dataset("v4/validation_int8.parquet", "validation.parquet")
    napi.download_dataset("v4/live_int8.parquet", "live.parquet")
```

データ分割: Purged K-Fold



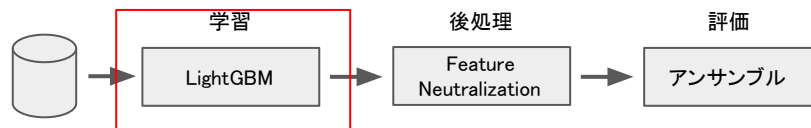
13

時系列データにおいて、**trainとtestの間に一定期間trainでもtestでもない間を開けたKFold**。時系列の変数に過去データを使うことが多いため、単純にk-foldするとleakしてしまう対策。



実際のExample scriptでは`get_time_series_cross_val_splits`関数が行っている

学習: LightGBM



14

LightGBMは勾配ブースティングを用いた機械学習フレームワーク^{*1}
パラメータ値について少し言及する

	Example Script	デフォルト値
n_estimators ^{*2}	2000	100
learning_rate	0.01	0.1
max_depth	5	-1
num_leaves	32	31
colsample_bytree	0.1	1.0

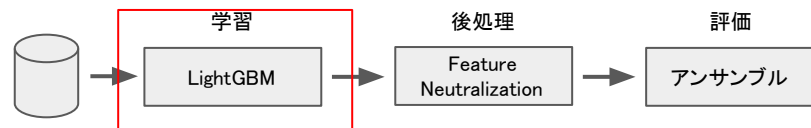
→ colsample_bytree = 0.1が珍しい

→ 1つの特徴量に依存しないことを良しとしているためと考えられる

*1: <https://lightgbm.readthedocs.io/>

*2: ちなみにEarly Stoppingはしてない

学習 : LightGBM (検証)



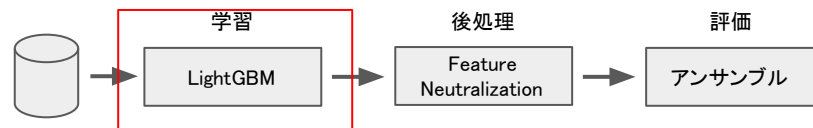
15

colsample_bytree=0.1と1.0で比較(その他条件はExample scriptと同じ)

(Sharpe ratio)	学習用データのCVスコア	
	colsample_bytree=0.1	colsample_bytree=1.0
nomi_20	1.69	1.47
nomi_60	1.72	1.60
jerome_20	1.51	1.48

→ colsample_bytreeを低くすることで、CVスコアの段階で有益な効果がありそう

学習: 複数の目的変数の利用



16

複数の変数に対して学習を行うことで精度がよくなることが知られている

→ 炉端会議^{*1}でも、Blendingに効くtargetがあることが示唆されてる

【Example script内での記載】

```
# randomly pick a handful of targets
# this can be vastly improved
targets = ["target", "target_nomi_v4_60", "target_jerome_v4_20"]
```

【炉端会議でのCEOコメント^{*1}】

もう1つリサーチしていることがあって、それは新しいターゲットについてだ。現状スコアリングに使われているターゲットは1つだが、それ以外にも色々なターゲットを提供している。スコアリングに使われているターゲットだけ使うのは典型的な間違いだ。他のターゲットにもっと情報がある。例えば、jerome_60はblendingによく効くターゲットだ。その他のターゲットを使うことで、しばしばオリジナルのターゲットで評価されるスコアも良くなるし、TCもそうだろう。ぜひ他のターゲットも考慮するモデリングをしてほしい。更に新しいターゲットのリリースも検討中だ。

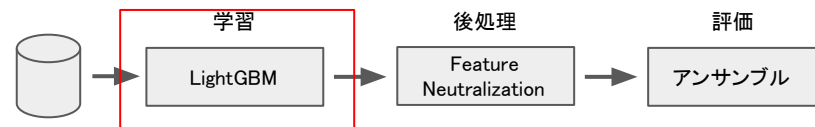
【Tips】

- ・20日後の結果を反映した_20系の変数と、60日後の結果を反映した_60系の変数がある
- ・_20系はtargetとの相関が0.64–0.86; _60系はtargetとの相関が0.37–0.48^{*2}

*1: <https://zenn.dev/katsu1110/articles/60c777d15e01d5>

*2: <https://zenn.dev/nishimoto/articles/2f1e2a8bfae84a>

検証: 複数の目的変数の利用



17

target以外の変数を予測した際のtargetとの相関平均、Sharpeを算出

*Validation data

【_20系ターゲット】

	Target	Mean	S.D.	Sharpe ratio
(target)	nomi_v4_20	0.023	0.031	0.760
	alan_v4_20	0.019	0.022	0.839
	arthur_v4_20	0.015	0.020	0.760
	ben_v4_20	0.018	0.027	0.675
	george_v4_20	0.009	0.018	0.472
	janet_v4_20	0.019	0.024	0.796
	jerome_v4_20	0.023	0.030	0.758
	paul_v4_20	0.009	0.018	0.535
	ralph_v4_20	0.022	0.030	0.729
	thomas_v4_20	0.018	0.027	0.648
	tyler_v4_20	0.023	0.031	0.738
	victor_v4_20	0.021	0.025	0.860
	waldo_v4_20	0.022	0.031	0.719
	william_v4_20	0.022	0.030	0.730

【_60系ターゲット】

	Target	Mean	S.D.	Sharpe ratio
(target)	nomi_v4_60	0.020	0.029	0.704
	alan_v4_60	0.017	0.022	0.760
	arthur_v4_60	0.014	0.020	0.729
	ben_v4_60	0.015	0.025	0.593
	george_v4_60	0.005	0.016	0.296
	janet_v4_60	0.017	0.024	0.701
	jerome_v4_60	0.020	0.027	0.763
	paul_v4_60	0.008	0.016	0.491
	ralph_v4_60	0.022	0.027	0.818
	thomas_v4_60	0.015	0.025	0.616
	tyler_v4_60	0.020	0.029	0.706
	victor_v4_60	0.019	0.023	0.816
	waldo_v4_60	0.022	0.028	0.782
	william_v4_60	0.020	0.026	0.780

(本来のtargetよりSharpe ratioが高い場合赤背景)

→ Blendingの素材とする他、予測すべきtargetを変更する戦略も考えられる

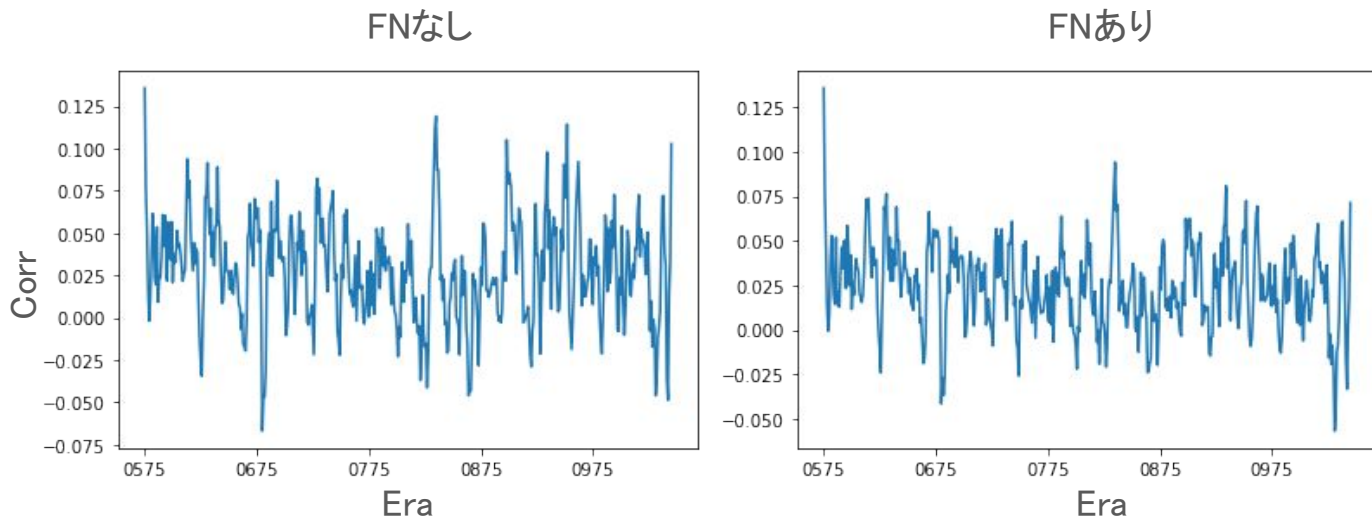
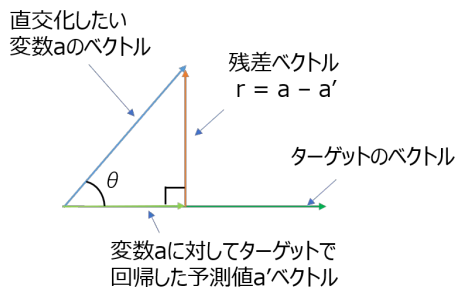
後処理: Feature Neutralization



Feature Neutralization (FN) とは？

→ **予測値**が特定の特数量に依存しないように後処理で変換する仕組み

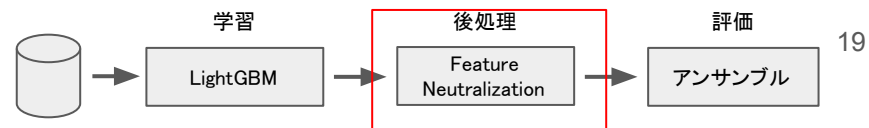
理論的な説明*1



→ FNをかけることで平均スコアは下がるが、スコアの標準偏差も下げることで結果を安定させることができる(Sharpe ratioの向上につながる)

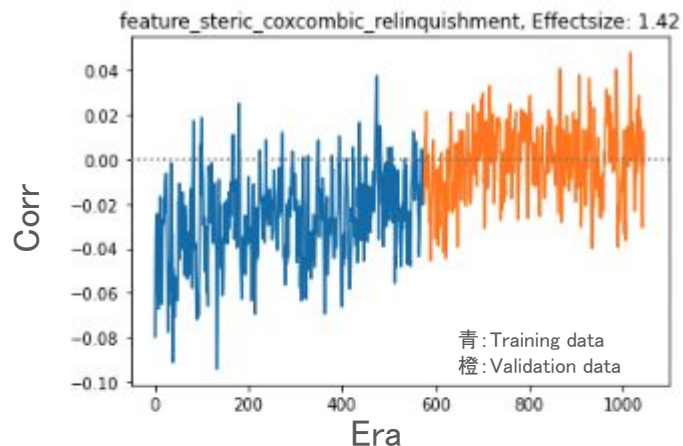
*1: <https://yaakublog.com/numerai-feature-neutralization>

後処理: 高リスク特徴量の同定



Numeraiが提供するいくつかの変数は危険なことが知られている*1
(元々は非常に高い相関を示したものの、今は相関がほぼゼロの変数など)

【相関が非常に悪くなっている変数の例】



【実際のソースコード】

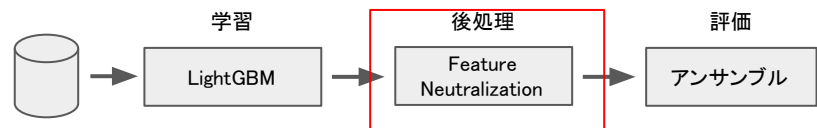
```
def get_biggest_change_features(corrs, n):  
    all_eras = corrs.index.sort_values()  
    h1_eras = all_eras[: len(all_eras) // 2]  
    h2_eras = all_eras[len(all_eras) // 2 :]  
  
    h1_corr_means = corrs.loc[h1_eras, :].mean()  
    h2_corr_means = corrs.loc[h2_eras, :].mean()  
  
    corr_diffs = h2_corr_means - h1_corr_means  
    worst_n = corr_diffs.abs().sort_values(ascending=False).head(n).index.tolist()  
    return worst_n
```

→ Example Scriptでは、学習データの前半期間と後半期間の平均値差分が大きいものを抽出

【Tips】高リスク特徴量は学習からそもそも除外する戦略もある(*1でも除外を推奨している)

*1: <https://forum.numer.ai/t/removing-dangerous-features/5627>

検証: Feature Neutralization



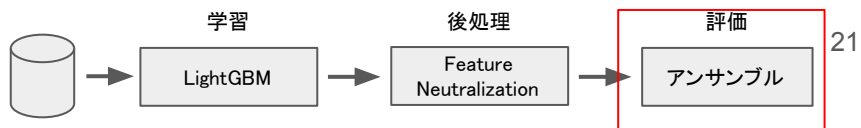
20

- ・どの特徴量に対しFNを適用するかは非常に難しい問題
- ・Example ScriptsではTop50の高リスク変数に対してFNを適用している
- ・全変数に対してFNを適用した際と、高リスク変数に対しFNを適用した際のSharpeを比較した

(Sharpe ratio)	学習用データの CVスコア		Validationデータのスコア	
	全変数	高リスク変数	全変数	高リスク変数
nomi_20	0.88	1.76	1.10	0.99
nomi_60	1.03	1.82	0.95	0.98
jerome_20	0.86	1.66	0.96	0.90

→ 学習用データでは高リスク特徴量に対してFNする方がよさそうだが、Validationデータでは同等程度となる

評価



これまで行ったことの結果を比較

Target	FN	学習用データの CVスコア		Validationデータのスコア	
		mean	sharpe	mean	sharpe
アンサンブル	o	0.044	1.941	0.027	0.993
アンサンブル	x	0.047	1.803	0.028	0.879
target	x	0.041	1.708	0.027	0.860
target	o	0.037	1.760	0.027	0.989
jerome_20	x	0.039	1.527	0.026	0.833
jerome_20	o	0.036	1.662	0.023	0.904
nomi_60	x	0.045	1.766	0.026	0.848
nomi_60	o	0.042	1.820	0.026	0.977

→ targetのアンサンブル+FNありが最も好成績を出す

まとめ

- Numerai Example Scriptは比較的シンプル
 - a. 特定の特徴量に依存しない仕組みが複数ある
 - colsample_bytreeを下げる
 - Feature Neutralization
 - b. ターゲットのアンサンブルによる精度ブースト
 - Example scriptは3ターゲットのアンサンブルを利用しているが、他の変数も検討の余地あり

Thank you!